Core Architectural Invariants

Purpose

Architectural Invariants define the properties of Shio Runtime that must never change.

Unlike Architectural Principles, which guide design decisions, Invariants describe conditions that are always expected to be true.

Violating an Invariant means violating the architecture itself.

These rules are considered permanent unless the architecture undergoes a deliberate redesign.

Invariant 1 — Character State Represents the Present

Character State always describes the current condition of the character.

It never represents:

- history;
- future intentions;
- completed actions;
- temporary events.

The present belongs to Character State.

Invariant 2 — Events Represent the Past

Events always describe something that has already happened.

Events are immutable.

Events are never modified after publication.

Historical facts never become Character State.

Invariant 3 — Every Responsibility Has Exactly One Owner

Every architectural responsibility belongs to exactly one Runtime.

Responsibilities must never overlap.

If ownership becomes ambiguous, the architecture should be reconsidered before implementation continues.

Invariant 4 — Only Action Runtime Changes the External World

No Runtime other than Action Runtime is permitted to produce observable side effects outside the Runtime.

All external changes must pass through:

- Tool Registry;
- Permission Guard;
- Action Runtime;
- External Adapter.

This boundary is absolute.

Invariant 5 — Adapters Never Contain Business Logic

Adapters translate.

Runtime systems decide.

Business logic never belongs inside Adapters.

Replacing an Adapter must never change character behavior.

Invariant 6 — Personality Is Independent From Technology

The identity of Shio does not belong to:

- the language model;
- the speech engine;
- the rendering engine;
- the storage provider.

Technologies express the character.

They do not define it.

Invariant 7 — Experience Always Follows Execution

Experience never performs actions.

It interprets completed actions.

Execution changes reality.

Experience changes the character.

These responsibilities remain permanently separated.

Invariant 8 — Runtime Systems Communicate Through Architecture

Runtime systems communicate only through architectural mechanisms.

Allowed:

- Character State;
- Event Bus;
- Public Contracts.

Hidden dependencies and direct implementation access are prohibited.

Invariant 9 — Technologies Are Replaceable

No technology is considered architecturally permanent.

Replacing an implementation should require Adapter changes rather than architectural redesign.

The architecture must outlive its technologies.

Invariant 10 — The Character Is More Important Than the Implementation

Every Runtime ultimately exists to strengthen the illusion of interacting with a believable digital character.

Whenever implementation quality and character quality conflict, preserving the character takes priority.

This invariant defines the ultimate purpose of the entire Runtime.

Final Statement

Architectural Principles explain how Shio Runtime should be designed.

Architectural Invariants define what must always remain true.

Together they establish the permanent foundation upon which the Runtime may continue evolving for years without losing its architectural identity.